TECHNICAL GUIDE · INTERNAL

Build the rewards program: Epos API + Shopify Admin API + Zapier

No monthly rewards-app fee — points run on Epos Now's own loyalty fields, Shopify's Admin API, and a Zapier automation as the connector. This is the concrete build plan, not the pitch page (that's in the meeting packet). Effort estimates assume you, not a client, doing the work with Claude/Cowork assisting on code.

Phase 1 — Get the Epos Now API credentials

~30 min

1. Log into the Epos Now Back Office for Michele's account.
2. Find **API Key** and **API Secret** under the developer/integrations settings (per-device — the token differs for every registered device/till).
3. Epos Now API uses **Basic Auth**, not OAuth: combine `KEY:SECRET` and Base64-encode it into an access token. Epos Now's developer portal has an encoder tool that does this for you.
4. Every request needs headers `Authorization: Basic <token>` and `Content-type: application/json`.
5. Store the key/secret in a password manager or environment variable — never in the repo or in chat.

Docs: developer.eposnowhq.com/Docs/Authentication · relevant resources: Customer (has native loyalty-points fields), Product, Transaction (V4 API — the V1/V2 Product endpoints are deprecated).

Phase 2 — Shopify Admin API access ALREADY DONE

0 hrs — done 9/25

The `be-you-tiful` custom app is installed on v0m0hb-x3 with all 18 scopes (including customers, orders, metafields/metaobjects), live on Railway at `app.be-you-tifulboutique.com`. Nothing left to configure here — this phase exists only so the plan is complete. Reference: `beau-bou-app/docs/SETUP.md` for the client-credentials token exchange if a script needs a fresh Admin API token.

Phase 3 — Design the points data model

~1–2 hrs

1. Decide where a customer's point balance lives on the Shopify side. Simplest: a `customer` metafield (namespace `rewards`, key `points_balance`, type `number_integer`) set via the Admin API. Avoids needing a database of your own since the custom app already has the DB, but a metafield is simpler to keep in sync and to display back to the customer.
2. Decide the earn/redeem rules with Michele (page 4 of the meeting packet: points per $1, points needed per reward, reward value, join bonus, birthday bonus, expiry).
3. Decide how a reward becomes a discount at checkout — the cleanest path is generating a single-use Shopify discount code via the Admin API once a customer redeems, rather than a running account credit.

Phase 4 — Wire the Zapier automation

~3–4 hrs

1. In Zapier, add the Epos Now app. Confirmed triggers available: **New Transaction**, **New Customer**, **New Product**, **New Customer Rating**, **New Booking**. Use **New Transaction** as the points-earning trigger.
2. Add a Formatter/Filter step: only continue if the transaction is a completed sale (skip refunds/voids), and calculate points earned from the sale total using Michele's chosen rate.
3. Add a step to look up (or create) the matching Shopify customer by email/phone — Epos Now's customer record and Shopify's customer record need to be matched, which is the trickiest part; the native Epos ↔ Shopify sync app (see the separate Product Sync Guide) keeping both customer lists aligned makes this reliable.
4. Add a Shopify action (or a webhook call into the `be-you-tiful` app's own API, which already has Admin API access) to increment the customer's `rewards.points_balance` metafield.
5. Build the mirror path for redemption: a Zap (or a small page in the custom app) that checks the balance, decrements it, and calls the Shopify Admin API to create a single-use discount code, then messages the customer the code.

Phase 5 — Customer-facing balance display

~2–3 hrs

- Simplest: a small section/block on the Shopify customer account page (theme edit in `beau-bou`) that reads the `rewards.points_balance` metafield and shows it — metafields can be exposed to the Storefront API/theme with a metafield definition marked storefront-visible.
- Stretch: a "redeem now" button that calls the custom app's endpoint directly instead of waiting on a Zap.

Phase 6 — Test end to end

~2–3 hrs

1. Ring a real test sale through Epos Now (small dollar amount), confirm the Zap fires and the Shopify metafield updates.
2. Redeem a reward, confirm the discount code works at checkout and the balance decrements.
3. Test the join bonus and (if in scope for launch) the birthday bonus logic.

Rough total: 10–16 hours of build + test time, once Michele's rules and Epos credentials are in hand. Phase 2 is already banked at $0.

This is genuinely more than a "point and click" app install — it's the trade-off for zero recurring app fees. If timeline pressure builds before Oct 15, Smile.io or Rivo (see the Rewards Options sheet) can go live same-day as a Phase 1 launch, with the custom build swapped in later without disrupting customers (their points would need a one-time migration in that case).